Pattern Recognition 124 (2022) 108423

Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/patcog

Improved time series clustering based on new geometric frameworks

Clément Péalat*, Guillaume Bouleux, Vincent Cheutet

Univ. Lyon, INSA Lyon, Université Claude Bernard Lyon 1, Université Lumière Lyon 2, EA 4570, DISP, Villeurbanne F-69621, France

ARTICLE INFO

Article history:
Received 2 November 2020
Revised 3 November 2021
Accepted 6 November 2021
Available online 9 November 2021

Keywords:
Clustering
Time series
Delayed coordinate embedding
Embedding
Stiefel manifold
UMAP
HDBSCAN

ABSTRACT

Most existing methods for time series clustering rely on distances calculated from the entire raw data using the Euclidean distance or Dynamic Time Warping distance. In this work, we propose to embed the time series onto higher-dimensional spaces to obtain geometric representations of the time series themselves. Particularly, the embedding on $\mathbb{R}^{n\times p}$, on the Stiefel manifold and on the unit Sphere are analyzed for their performances with respect to several yet well-known clustering algorithms. The gain brought by the geometrical representation for the time series clustering is illustrated through a large benchmark of databases. We particularly exhibit that, firstly, the embedding of the time series on higher dimensional spaces gives better results than classical approaches and, secondly, that the embedding on the Stiefel manifold - in conjunction with UMAP and HDBSCAN clustering algorithms - is the recommended framework for time series clustering.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Clustering is one of the most famous unsupervised machine learning methods [1]. This set of data mining techniques gives insight of the structure of the data. The goal is to determine from data with multiple unlabeled observations, groups (clusters) of elements with the closest behaviour. In order to do that, the clustering algorithms create clusters aiming to maximize similarities for the elements inside the same cluster while minimizing similarities between elements of different clusters. The similarities obtained are closely related to the specific characteristics of the data. The databases studied in this work are one-dimensional time series. This data structure is studied in several domains such as mechanical diagnosis [2], healthcare [3,4] and many others [5]. The time series clustering corresponds to the application of the clustering methods to this special kind of data. One way to do it is to directly apply clustering algorithms to the raw data [6]. Most of the studies that work directly on raw data use Dynamic Time Warping distance [7] or Euclidean distance. But, in some cases, it does not give accurate results, as it was the case in one of our precedent works on medical data [8]. A potential cause of why the performances are not as expected may lie in the fact that these metrics focus on local similarities on the raw data while forgetting the hidden global dynamic of the time series.

Some works have studied clustering with other distances but from the angle of high-dimensional time series. For example, the motion of a rigid body over time [9,10] which is characterized by a vector of $\mathbb{R}^3$ evolving over time, can be modeled by high-dimensional time series. In this case, it has been demonstrated in the literature that manifold learning and subspace-based clustering methods give both an accurate description of the data and an improved clustering with respect to Euclidean distances [10–15]. If the literature is vast concerning the clustering of high-dimensional data, such as clustering of images, or rigid body trajectories to name a few, there is a huge gap applying manifold-based approaches for one-dimensional data clustering. For example, those reviews on one-dimensional time series clustering [16,17] do not address these approaches at all. We only find it in [18]. Consequently, the present work explores and evaluates some well-known manifold-based approaches in the case of one-dimensional time series and aims to bridge the gap between one-dimensional data and their higher-order geometrical features, useful for improving the clustering.

Extracting intrinsic geometrical properties from a time series can be processed in different ways. We will quote for example the class of works that uses second order statistics [19], the class of works that proposes a direct description of the time series on a Lie group [20,21] or finally the class of works that uses the famous delay coordinate embedding [22] which gives a description of the time series in the phase space. Obviously these classes do not represent the exhaustiveness of the approaches for the geometrical characterization of time series, but the references given may help the reader, or at least help him to start, to enrich his state of the art.

* Corresponding author.
E-mail addresses: clement.pealat@insa-lyon.fr (C. Péalat), guillaume.bouleux@insa-lyon.fr (G. Bouleux), vincent.cheutet@insa-lyon.fr (V. Cheutet).

https://doi.org/10.1016/j.patcog.2021.108423
0031-3203/© 2021 Elsevier Ltd. All rights reserved.

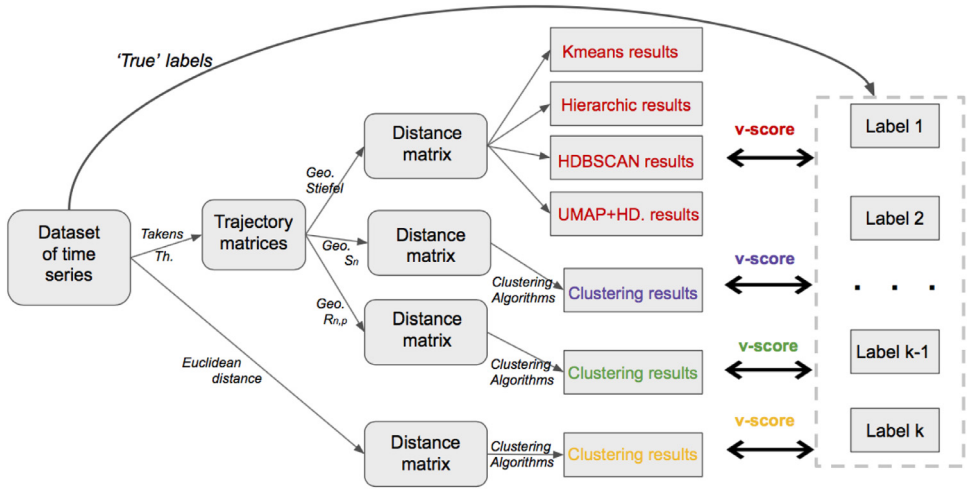


Fig. 1. Summary of our benchmark. For a database, 16 clustering results are obtained depending on both the geometrical representation and the clustering algorithm. Then, using the v-measure score, those results are compared to the expected labels.

In this work, we want to cluster time series via the extraction of geometrical characteristics obtained by the delay coordinate embedding. This so-familiar embedding is intimately relied with the Takens' theorem. The idea behind this theorem [22] is to consider a time series as a measure of a hidden dynamic system of larger dimension. It states that it is possible to recover an accurate representation of the hidden system once some parameters, the delay and the dimension embedding, have been estimated. Embedding a time series in such a way leads to describing the time series by its trajectory on the phase space. It allows to have a grasp of the topological structure of the data through the attractor of the system, and so hidden but intrinsic features for the time series. Then, we are more looking for global qualitative information rather than local quantitative information.

The induced geometry of the system (of the trajectory) may be therefore studied with several methods [19,23] by considering the matrix associated with the trajectory. In order to fully exploit the geometrical information brought by those matrices, we propose here to project them onto three manifolds, directly $\mathbb{R}^{n\times p}$, the Stiefel manifold and the unit Sphere. This way, well-known clustering algorithms can be adapted to account for the distance defined on these manifolds. In particular, we propose to compare the clustering performances of the Agglomerative Hierarchical algorithm, the K-means algorithm and, based on the good results obtained in [24], we have also decided to test the HDBSCAN algorithm [25] enhanced by UMAP [26]. Those algorithms are all tested on the three manifolds proposed.

Our point of view being to use the geometrical information of time series trajectories obtained by the delay coordinate embedding, we have first tested the relevance of such a hypothesis through time series generated from dynamic systems well known in the literature. Afterwards, we have tested all the couples (manifold-type, clustering algorithm) on 79 databases available in the UCR time series classification archive, where the 'true' labels of the time series are known. This process is summarized in Fig. 1.

The paper organizes as follow. In Section 2, we present the delay coordinate embedding and the resultant embedding into the manifolds. Section 3 introduces important notions of Riemannian geometry that allows to determine the distances as well as the Karcher means of each manifolds. In Section 4, the clustering algorithms are presented in detailed and in Section 5, we present the clustering results obtained on known time series models. Next, an exhaustive analysis of the frameworks proposed in this work are displayed in Section 6 with the application of the frameworks on the 'UCR time series'. Finally, a conclusion as well as elements of perspectives are given in the Section 7.

Notations

D is a database of time series of length l: $D = (Y_i(t), t = 0, \dots l-1, i = 0, \dots, n-1)$, with each time series characterised by a label.

- $O(n)$ is the orthogonal group of dimension n defined by $O(n) = \{A \in \mathbb{R}^{n\times n} : A^TA = I_n\}$ with I_n the identity matrix of dimension n.
- $V_{n,p} = O(n)/O(n-p)$ is the Stiefel manifold. It is a subspace of $\mathbb{R}^{n\times p}$ defined by $V_{n,p} = \{A \in \mathbb{R}^{n\times p} : A^TA = I_p\}$
- S_n is the unit Sphere of $\mathbb{R}^n$.
- T_XM is the tangent space at X of the manifold M

2. Representation manifold

One-dimensional time series can be seen as a discrete measurement (observation) of a dynamic system. The underlying dynamic system is in most situations of large dimension, obviously unknown at first glance. Takens' theorem [22] states that it is possible, from this observation, to reconstruct the trajectory (the attractor) of the dynamic system by a basic delay coordinate embedding of the time series. From a time series $y = (y(0), \dots, y(l-1))$, we subdivide it into vectors of size m defined by $(y(k), y(k+\tau), \dots, y(k+(m-1)\tau))$, $k = 0, \dots, l-1-(m-1)\tau$. By concatenating those vectors for all k, we obtained a trajectory matrix [19,27] T_y of the time series y defined as:

$$T_y = \begin{bmatrix} y(0) & y(\tau) & \cdots & y((m-1)\tau) \\ y(1) & y(2+\tau) & & \vdots \\ \vdots & \vdots & & \vdots \\ y(k) & y(k+\tau) & \cdots & y(k+(m-1)\tau) \\ \vdots & \vdots & & \vdots \\ y(l-1-(m-1)\tau) & y(l-1-(m-2)\tau) & \cdots & y(l-1) \end{bmatrix} \tag{1}$$

This representation of the time series through a time-delay embedding allows to better determine an hidden topological feature of it. It determines the attractor of the dynamical system hidden behind the time series. Working with delay coordinate embedding

allows then to compare directly the attractor instead of the one-dimensional measure. Each row of the trajectory matrix represents a state of the time series in the embedded space which is now of larger dimension. Moreover, the trajectory matrix is an element of $\mathbb{R}^{n\times p}$ with $n = l - 1 - (m-1)\tau$ and $p = m$. From the previous relation, it is clear how the two parameters, m, the dimension of embedding and τ, standing for the delay, have a great importance in the geometric characterisation of the attractor for the time series.

Once the trajectory matrices have been determined from the time series, several possibilities exist to treat those elements of $\mathbb{R}^{n\times p}$. We studied here three geometric representations: a direct one ($\mathbb{R}^{n\times p}$), the Stiefel manifold, and the unit Sphere which allows to deal with Stiefel manifolds of different dimensions.

By treating directly the trajectory matrix T_y of (1) as an element of $\mathbb{R}^{n\times p}$ leads to straightforwardly consider the Frobenius norm defined by the following :

$$d(A,B) = \left(\sum_{k=0}^{n-1}\sum_{l=0}^{p-1}(a_{k,l} - b_{k,l})^2\right)^{\frac{1}{2}} \ \forall\, A, B \in \ \mathbb{R}^{n\times p}.$$

as the distance between the different trajectory matrices. In this context, the mean admits the basic expression $\overline{A} = \frac{1}{k}\sum_{i=1}^{k} A_i, \forall\, A_1, \ldots, A_k \ \in \mathbb{R}^{n\times p}$.

If we now go a little further in the information carried by the trajectory matrix, instead of using the Euclidean distance between the matrices, *i.e.* the Frobenius distance, we propose to measure the distance between the trajectory matrices by the distance of the subspace they span. This is equivalent to projecting the trajectory matrices on the Stiefel manifold.

The Stiefel manifold is a subspace of $\mathbb{R}^{n\times p}$ defined by $V_{n,p} = \{A \in \mathbb{R}^{n\times p} : A^T A = I_p\}$. For sufficiently small values of m and τ, we have directly $p < n$. It is thus enough to orthogonalize, via a QR decomposition, the trajectory matrices to see them as elements of $V_{n,p}$. We precise that we only dealt with the $p < n$ case. However, we can note that in the $p > n$ case, it is possible to apply a dimension reduction to return to $p < n$ [8]. So far we have deliberately not discussed the values of m and τ, which we have implicitly assumed to be equal for each of the embedded time series (inside a database). Of course, in order to make the best use of the geometric information of the time series attractor, these values may vary and thus be different for each of the trajectory matrices to be compared. To cope with this, the trajectory matrices can be projected onto the unit Sphere [28]. Let $U_1, U_2, \ldots, U_k$ be elements of $(V_{n,p_1} \times V_{n,p_2} \ldots \times V_{n,p_k})$. To compare those elements, we compute the projection matrix $P_{U_i} = U_i U_i^T$ which then becomes an element of $\mathbb{R}^{n\times n}$ with the particularity to be a symmetric matrix. This matrix is then embedded as a vector of $\mathbb{R}^{n(n+1)/2}$ composed of all the different elements of the symmetric matrix only once. To fix the ideas, in $\mathbb{R}^{2\times 2}$, we have:

$$\begin{pmatrix} a & b \\ b & d \end{pmatrix} \rightarrow \begin{pmatrix} a \\ b \\ d \end{pmatrix}. \tag{2}$$

Thanks to the orthogonality of the matrix $U_i, i = 0, \ldots, k$, all the associated vector v_i of $\mathbb{R}^{(n(n+1)/2)}$ are on the same Sphere of a radius r and center c in $\mathbb{R}^{(n(n+1)/2)}$. To simplify, the vector on the unit Sphere of $\mathbb{R}^{(n(n+1)/2)}$ is put back using $v_i' = \frac{v_i - c}{||v_i - c||}$. Thus, we can determine the distance and the mean using the geometry of $S_{(n(n+1)/2)}$. This will be discussed in Section 3.

The Stiefel manifold and the unit Sphere are particular spaces. They are not flat spaces like the Euclidean space is. The distance and the mean with respect to these manifolds is determined via the Riemannian geometry.

Delay coordinate embedding is a classical method to analyze time series. However, it is difficult to find in the literature a geometrical representation that gives satisfaction. We hope that here by testing on a large benchmark we highlight the interest of addressing this issue because it can profoundly impact the quality of results.

3. Geometries of the proposed manifolds

3.1. Riemannian geometry

A manifold $\mathcal{M}$ is a curved space that can be seen locally as an Euclidean space. We denote as $C(\mathcal{M})$ the set of curves on $\mathcal{M}$ and g the metric. (M, g) is a Riemannian manifold. A distance can be defined with respects to this metric. Let's call $\gamma : [a; b] \rightarrow M$ a smooth curve on the manifold (M, g). The length of this curve is defined by $L(\gamma) = \int_a^b \sqrt{g(\gamma'(t), \gamma'(t)}\, \mathrm{d}t = \int_a^b ||\gamma'(t)||\, \mathrm{d}t$. Let X,Y be two elements of the manifold M. The distance between X and Y is then defined by:

$$\forall X, Y \in M, d_M(X, Y) = \min_{\gamma\in C(\mathcal{M}), \gamma(0)=X, \gamma(1)=Y} L(\gamma) \tag{3}$$

The curve γ realizing the minimization is called the geodesic.

3.2. Karcher mean

Once the distance is defined, a mean with respect to this distance, must be also defined. It is the Karcher mean, defined by:

$$\overline{U} = \underset{X\in M}{\operatorname{argmin}} \sum_{i=1}^{n} d_M(X, U_i)^2, \ \forall\, U_1, \ldots U_n \in M. \tag{4}$$

To determine this mean, we can use a gradient descent algorithm. To do so, we derive the Eq. (4), as proposed by Karcher [29], and we obtain a new definition for the Karcher mean $\overline{X}$:

$$\sum_{i=1}^{n} Log_{\overline{X}}(X_i) = 0, \ \forall X_1, \ldots X_n \in M. \tag{5}$$

This equation uses the Riemannian logarithm, the inverse function of the Riemannian exponential.

With this Eq. (5), we can deduce our gradient descent algorithm (see Algorithm 1).

Algorithm 1: Karcher means.

Input: $X_1, X_2, \ldots X_N \in M$;
$\mu_0 = X_1$
while $\mu_k \neq \mu_{k+1}$ **do**
  $\Delta\mu = \frac{1}{N}\sum_{i=1}^{N} Log_{\mu_k}(X_i)$;
  $\mu_{k+1} = Exp_{\mu_k}(\Delta\mu)$;
end
Output: μ

3.3. Geometry of the Stiefel manifold

On the Stiefel manifold, the geodesic distance is given by the principal angles of their subspaces. For any $A, B \in V_{n,p}$, we have:

$$distance(A, B) = \left(\sum_{i}^{d} \theta_i^2\right)^{\frac{1}{2}} \tag{6}$$

Those angles determine a path on the Stiefel manifold with a succession of continuous rotation. Figure 2 illustrates an example of two elements A, B of $V_2(\mathbb{R}^3)$. A base of the range-space $\mathcal{R}(A)$ (which is spanned by the columns of A) is defined by the two vectors a_1, a_2 in green and a base of $\mathcal{R}(B)$ is defined by the two vectors b_1, b_2 in red. A double continuous rotation of $\theta_1 = (a_1, b_1)$ and $\theta_2 = (a_2, b_2)$ determines a path from A to B on the manifold.

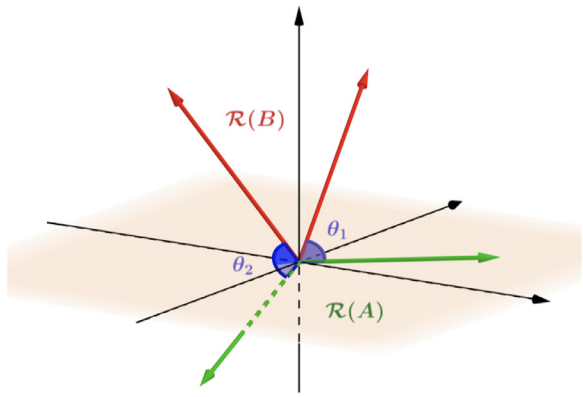


Fig. 2. Example of Principal Angle between two elements A and B of $V_3(\mathbb{R}^2)$. A and B are each generated by two vectors. Only two rotations are needed to go from A to B.

But this is not necessarily the shortest path. A double rotation of angle $\theta_1' = (a_1, b_2)$ and $\theta_2' = (a_2, b_1)$ also determines a path from A to B on the manifold. To ensure that we deal with principal angles, we look for the rotations of smallest energy. To do so, between two elements A, B of $V_{n,p}$ [30], we compute the singular values $\lambda_i, i = 1, ., p$ of $A^T B$. Then, for i in $1, \dots p$, $\theta_i = \arccos(\lambda_i)$ are the principal angles between A and B. The quick Algorithm 2 gives

Algorithm 2: Computation distance on the Stiefel manifold.

Input: $A, B \in V_{n,p}$
$A^T B = U \Sigma V$ (Singular Value Decomposition)
$\theta_i = \arccos(\lambda_i), i = 1, \dots, p$
Output: $d(A, B) = \left(\sum_{i=1}^{p} \theta_i^2\right)^{\frac{1}{2}}$

the computational point of view of the distance on the Stiefel manifold.

Using this distance, we can apply Agglomerative Hierarchical clustering, HDBSCAN and UMAP algorithms, but we need the associated average to use the K-means algorithm. To do so, we need to compute the Riemannian logarithm and the Riemannian exponential on the Stiefel manifold. In [31], two algorithms are proposed to compute those two functions.

3.4. Geometry of S_n

The unit Sphere [32] on $\mathbb{R}^n$ is defined by $S_n = \{X \in \mathbb{R}^n : X^T X = 1\}$. The distance between X, Y of S_n is defined by:

$$d(X, Y) = arccos(X^T Y)$$

We can determine the mean on the Sphere using the same algorithm that we use for the Stiefel manifold. To do so, we need to determine the Riemannian exponential and the Riemannian logarithm. Their definition is proposed on Table 1 that summaries the different elements needed to apply the clustering algorithms for each manifold.

4. Clustering algorithms and scores

Once the distances and the means on the different geometrical representations have been determined, clustering algorithms are applicable. To determine the clusters, we proposed to use HDBSCAN (density-based clustering algorithm) [25], HDBSCAN enhanced by UMAP [26], K-means and Agglomerative Hierarchical clustering.

K-means and Agglomerative Hierarchical Clustering: For both of those clustering algorithms, the number of clusters needs to be determined beforehand. To do so, we propose to use the Silhouette score [33]. We keep the number of clusters that maximizes this indicator. All these methods are commonly used and have been proven to work. They serve as a basis for comparison in our benchmark.

HDBSCAN: To complete the tested clustering algorithms, we have opted for a density-based clustering algorithm: HDBSCAN, a prolongation of Density-Based Spatial Clustering of Applications with noise (DBSCAN) [34,35].

All the parameters for this algorithm have been left at default (values visible at https://hdbscan.readthedocs.io/en/latest/index.html).

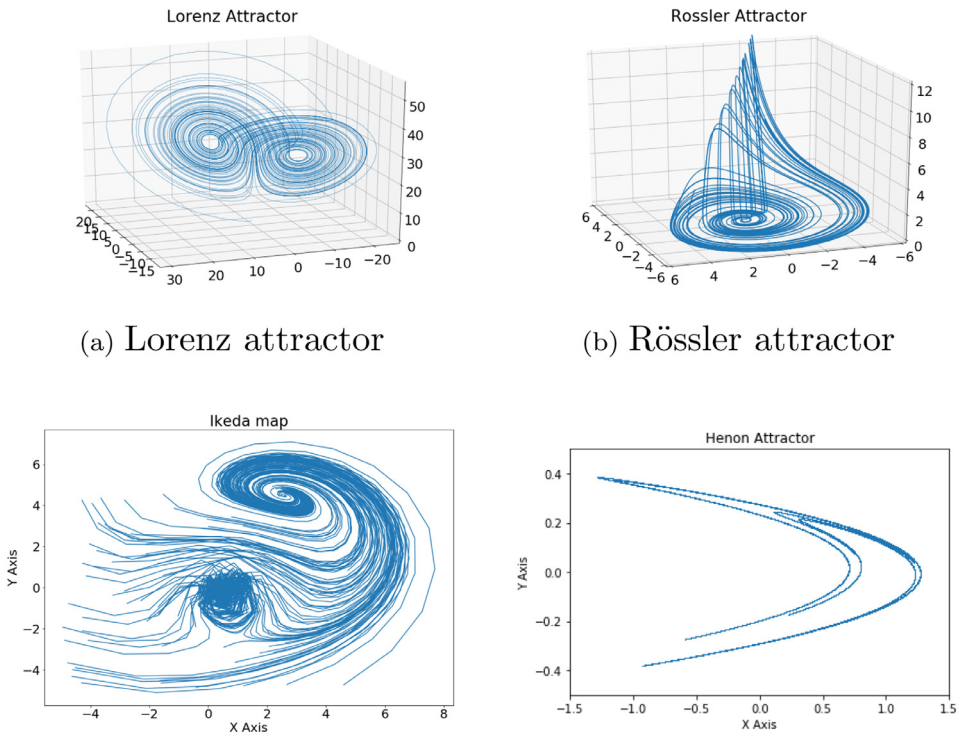
HDBSCAN enhanced by UMAP: To increase the quality of the density algorithm of clustering, we use beforehand Uniform Manifold Approximation and Projection (UMAP) [26]. This decision is motivated by the fact that in some cases the density is not clear enough to use directly HDBSCAN. Indeed, in a lot of our applications, lot of elements of the data are considered as noise. The clustering algorithm decides that they are errors in the database. So, UMAP allows, by embedding in a lower dimension the data, to increase the gap between groups. There are still some concerns with this method because UMAP does not completely keep the density of the data. So, it can create pseudo-groups inside one 'true' cluster. However, in [24], we have shown that a reduction of dimension through UMAP improved the clustering performances of the HDBSCAN algorithm on time series from real data. Using the databases available at www.cs.ucr.edu/~eamonn/time_series_data, we compared the clustering results of 6 clustering algorithms on 85 databases and the combination UMAP + HDBSCAN has clearly shown better results. UMAP realizes a reduction of dimension which clarifies the structure of the data for HDBSCAN. For example, in the case of the embedding into Stiefel manifold $V_{n,p}$, the time series belongs to an ambient space of dimension $d_{stiefel} = pn - \frac{p(p+1)}{2}$. So, UMAP realizes a reduction dimension from $d_{stiefel}$ to a selected dimension.

To realize the clustering, a global view of the structure of the data is needed. So, we fixed the parameters of the number of neighbors (to determine the reduction of dimension) at 30 throughout the rest of the article (or 10 if the concerned database has not enough samples) and the dimension of reduction is fixed at 10 (values proposed on https://umap-learn.readthedocs.io/en/latest/).

The use of the framework UMAP + HDBSCAN is to our knowledge innovative and at the time this research has been proposed we did not find any article validating the effectiveness of this framework. For example, the review of clustering algorithms pro-

Table 1
Different computational formulas for each manifold.

Geometry	$\mathbf{V_{n,p}}$	$\mathbf{S_n}$	$\mathbb{R}^{\mathbf{n\times p}}$
Distance	See algo 2	$d(X, Y) = \arccos(X^T P)$	$d(X, Y) = (\sum_{k=0}^{n-1} \sum_{l=0}^{p-1} (x_{k,l} - y_{k,l})^2)^{\frac{1}{2}}$
Karcher mean	Gradient descent algorithm (Algorithm 1)		$X_1, \dots, X_n \in \mathbb{R}^{n\times p}, \overline{X} = \frac{1}{k} \sum_{i=1}^{k} X_i$
Riemannian exponential	See algo in [31]	$Exp_X(v) = \cos(\lvert v\rvert) \times x + \frac{\sin(\lvert v\rvert)}{\lvert v\rvert} \times y$	$Exp_X(v) = X + v$
Riemannian logarithm	See algo in [31]	$Log_X(Y) = \frac{d(X,Y)}{\sqrt{1-(X^T Y)^2}} (Y - (X^T P)X$	$Log_X(Y) = Y - X$



(a) Lorenz attractor (b) Rössler attractor

(c) Ikeda Map, trajectories of 100 dots in $\mathbb{R}^2$ (d) Hénon Map, trajectories of 100 dots in $\mathbb{R}^2$

Fig. 3. Representation of the four dynamic systems.

posed in [36] does not mention HDBSCAN for the group of density algorithms, let alone its use with UMAP. This seemed like a great addition to our benchmark.

Evaluation of the clustering: Once the clustering results have been computed, indicators are needed to determine the quality of the results compared to the 'true' labels. The relevance indicator we chose was the v-measure score [37]. It is an entropy-based clustering evaluation which realizes a harmonic mean between the completeness and homogeneity of the expected labels (true clusters) and the clustering results. An other possible indicator is the Adjusted Rand Index Score (ARI) [38]. It is an extension of the Rand Index Score (RI). ARI takes values between -1 and 1 (with 1 a perfect clustering) and ensures that a random results has a value of 0.

5. Clustering results from known models

To get a first glimpse of the efficiency of the framework proposed in this work, we used four known dynamic systems from which we selected one coordinate to obtain a time series. We used:

- The Lorenz system. It is plotted in Fig. 3a. This figure is the trajectory of a dot submitted by the equations of the Lorenz system.
- The Rössler system. It is a dynamic system of $\mathbb{R}^3$. The movement is determined by a set of 3 coupled differential equations. We plotted the movement of a dot on the Fig. 3b.
- The Henon system. This dynamic system is a two-dimensional system, and we traced the trajectory of 100 dots in Fig. 3d.
- The Ikeda system. It is initially a discrete-time dynamical system on the complex map. A 2D example of 100 dots is given on Fig. 3c.

We used next these dynamic systems for creating known time series clusters.

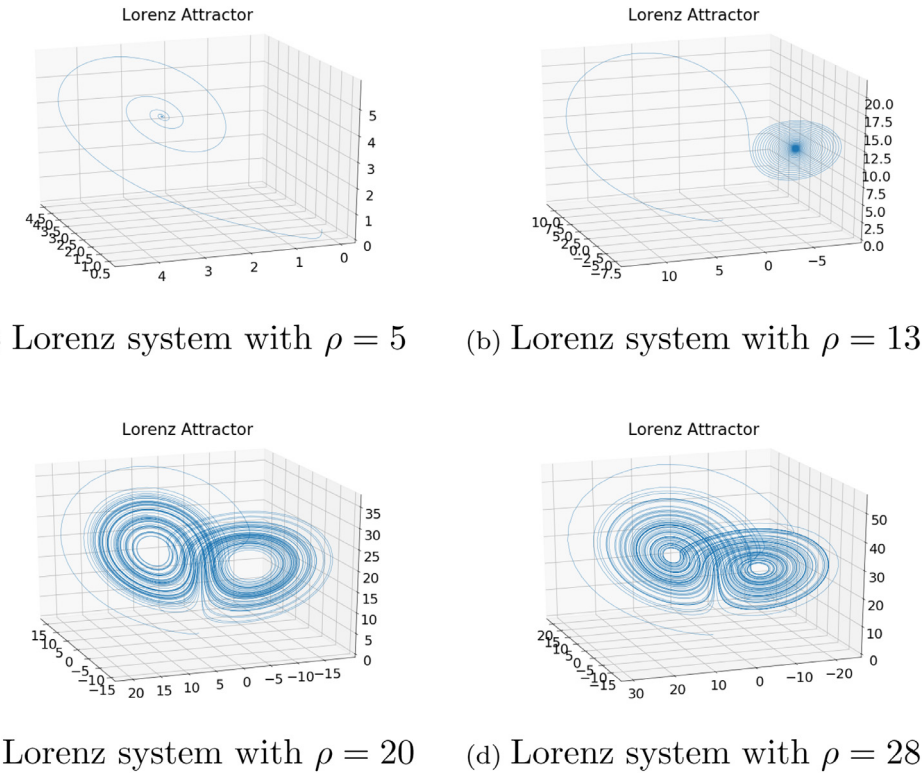
5.1. Clustering results from a unique known model

In this scenario, we propose to create a first database from the Lorenz system. On the Lorenz' equation, an important parameter is ρ. Depending on the values of this parameter ρ, the trajectory obtained by the Lorenz equations is very different. In the Fig. 4, we have plotted four trajectories got from four different values of ρ, the other parameters remaining unchanged. For each value of ρ, 100 trajectories according to the coordinates x, y and z have been calculated. For that, the differential equations have been discretized with a time step of 0.01. The difference between the trajectories comes from the initial conditions, randomly chosen between 0 and 1. Then, we kept only the x component and we therefore built 400 one-dimensional time series each belonging to one of the four known cluster.

The delay parameter τ used for the delay coordinate embedding was kept to the value 10 and we fixed m at the value 3. Then, the embedding into the Stiefel manifold, via the QR decomposition, into the unit Sphere and into $\mathbb{R}^{n\times p}$ are computed as well as the distance between the time series with respect to the geometrical representations. Finally, the clustering was done by the UMAP and HDBSCAN clustering algorithm.

To check the quality of the results, we compared the results of the clustering to the 'true' clusters generated.

For the scenario studied, we obtained a v-measure score of 1 for the Stiefel and unit Sphere manifolds embedding of the trajectory matrices and a v-measure score of 0.721 when trajectory matrices are embedded in $\mathbb{R}^{n\times p}$. The lowest performance was obtained by using the Euclidean distance between the time series with a v-measure score of 0.648. For this toy example, we thus conclude



(a) Lorenz system with $\rho = 5$ (b) Lorenz system with $\rho = 13$

(c) Lorenz system with $\rho = 20$ (d) Lorenz system with $\rho = 28$

Fig. 4. Trajectory of the Lorenz system for several values of ρ.

that the Stiefel and the unit Sphere embedding seems to completely exploit the geometrical intrinsic properties of the attractor of the time series, giving in conclusion the best performances with no error at all.

5.2. Clustering results across several known models

As we did in the previous section, we created a database of time series. For that scenario, the time series are divided into four clusters according to the dynamic system from which they are obtained. We computed for each dynamic system 100 trajectories with the same parameters as given previously, and the initial conditions randomly chosen between 0 and 1. From this, we extracted the x coordinate of the trajectories. It gave us a database of 400 one-dimensional time series evenly distributed between four clusters. Alike the previous scenario, the parameters used for the Lorenz system are fixed to $\tau = 10$ and m=3. The delay parameter for the Ikeda and Henon systems is fixed to $\tau = 1$, whereas the delay parameter for the Rössler system is fixed to $\tau = 100$. The embedding parameter m is kept equal to 3 for all the systems. The fixed parameters given have been estimated to fully retrieve the geometrical properties of the associated attractors. We can notice that since the values of τ are not the same across all the databases, the trajectory matrices are of different dimensions, indeed $p = m$ and $n = l - (m-1)\tau$, $l = 5000$. Consequently, we have decided to keep only the first $l - 1 - (m-1)100$ vectors of $\mathbb{R}^m$. In this way, all the matrices will belong to the same space $\mathbb{R}^{n\times p}$. We obtained a v-measure score of 0.8456 for the Stiefel manifold, a v-measure score of 0.837 for S_n, of 0.471 for $\mathbb{R}^{n\times p}$ and once again, the worst clustering performance was obtained by the Euclidean distance on the raw data with a v-measure score of 0.460.

This second scenario confirms the good capacities of the framework composed by a delay coordinate embedding followed by an embedding of the trajectory matrices on the Stiefel manifold to cluster the time series. Even if the clustering results are less exceptional than the previous scenario, a v-measure score of near 0.85 is significant. Clearly this preprocessing framework, as the first stage before employing clustering algorithms, shows the importance of the intrinsic geometrical property of the time series for the clustering. But, shows that the representation manifold of the time series attractors needs to be carefully chosen as well.

Table 2
Presentation of the databases.

Number of databases	79		
	Min	Mean	Max
Nb of clusters	2	7.5	60
Nb of time series	16	432	8926
Length of time series	24	422	2709

6. Clustering results on UCR time series classification archive databases

To realize our benchmark, we used the 79 databases available at UCR time series classification archive. A brief summary of those databases are presented in Table 2. The number of clusters in a database varies from two (only two labels) to 60. Moreover, some databases have a large number of time series to cluster whereas others have a weaker one. For a given database, all its time series have the same length. The length of the time series varies from 24 samples to a maximum of 2709 samples.

Table 3
Main characteristics of the v-measure score obtained. For 4 geometries and 4 clustering algorithms, this table gives the mean and the standard deviation along the 79 databases. Also, the number and percentage of databases for which a method is the best are given. The same elements but for the ARI score are in brackets.

Geometry	$\mathbb{R}^{n\times p}$				S_n			
Clustering algo.	K-means	Hier.	HDB.	U.+HDB.	K-means	Hier.	HDB.	U.+HDB.
Mean	0.240 (.220)	0.224 (.152)	0.217 (.151)	0.300 (.226)	0.303 (.228)	0.254 (.199)	0.227 (.178)	0.322 (.252)
Std	0.241 (.252)	0.253 (.208)	0.232 (.202)	0.272 (.234)	0.260 (.235)	0.290 (.270)	0.250 (.232)	0.270 (.230)
Times best results	3 (4)	2 (3)	3 (4)	9 (7)	7 (8)	10 (5)	6 (6)	12 (6)
% best results	3.8% (5.1)	2.5% (3.8)	3.8% (5.1)	11.4% (8.9)	8.9% (10.1)	12.7% (6.3)	7.6% (7.6)	15.2% (7.6)
Geometry	$V_{n.p}$				Euclidean, $\mathbb{R}^l$			
Clustering algo.	K-means	Hier.	HDB.	U.+HDB.	K-means	Hier.	HDB.	U.+HDB.
Mean	0.245 (.166)	0.220 (.223)	0.190 (.164)	0.376 (.270)	0.281 (.210)	0.242 (.138)	0.215 (.131)	0.293 (.183)
Std	0.228 (.224)	0.263 (.265)	0.219 (.224)	0.271 (.242)	0.256 (.228)	0.260 (.219)	0.233 (.201)	0.259 (.217)
Times best results	7 (4)	5 (5)	4 (4)	21 (14)	8 (8)	6 (0)	3 (0)	4 (1)
% best results	8.9% (5.1)	6.3% (6.3)	5.1% (5.1)	26.6% (5.1)	17.7% (10.1)	7.6% (0)	3.8% (0)	5.1% (1.3)

6.1. How we did it

We recall and specify our methodology here to create the benchmark. From open access databases, we have compared 4 different geometries and used 4 clustering algorithms. This gave us 16 different clustering results per database which we then compared to the expected labels (see Fig. 1). Three of our geometries were linked to the delay coordinate embedding. The time series were then transformed into a trajectory matrix which depended on two parameters m and τ (as already exposed in the previous sections). After some exploratory works, we have found that the embedding parameter m was not determinant and could be kept to a fixed value we chose equal to 5. On contrary, we found that the delay parameter τ was important for the clustering. To determine it, we have decided to use the real database labels. We have varied the values of τ between 1 and 20 and computed the clustering results between each elements for each geometry. Then, from those results, the v-measure score have been deduced with the true labels. For each of the three geometries, the τ values which maximized the v-measure score was kept.

6.2. Analysis of the clustering results

On Table 3, we present:

- The mean of the v-measure score for each method of clustering across the 79 databases
- The standard deviation of the v-measure score
- The number of databases for which a method is the best one. For example, K-means on $\mathbb{R}^{n\times p}$ has the best results for 3 databases (among 79 databases). We can notice that in some cases, the best score for a database is obtained by several methods.

We present also in brackets the same elements but with the ARI score. The complete results are available in the appendix (see Table A.1, A.2 and A.3).

For the rest of the analysis, we focus on the v-measure score. Analysing the results, the best method is clearly the application of UMAP and HDBSCAN framework on the Stiefel manifold. Indeed, it presents the best average v-measure score (0.376) and gives the best results for 21 databases out of 79. It is quite clearly superior to the second best average of 0.322 obtained for the combination UMAP + HDBSCAN on the unit Sphere which is the best method for 12 out of 79 databases. Overall, the other methods propose sightly inferior mean values, between 0.2 and 0.3.

From the point of view of the geometric representation, we have:

- the Stiefel manifold which is the best geometrical representation for 46.9% of the databases. This means that for 37 databases across the 79, one of the 4 clustering algorithms used on the Stiefel manifold gives the best results.
- The unit Sphere which is the best for 44.3% of the databases (35/79).
- The space $\mathbb{R}^{n\times p}$ which is the best for 21.5% of the databases (17/79).
- The Euclidean on raw data which arrives at the last place with only 26.6% of databases well clustered (21/79).

Euclidean on raw data is clearly a less efficient method than the metrics induced by the Stiefel manifold or the unit Sphere representations. It validates once again the interest of using the delay coordinate embedding to obtain geometric features of the time series through the trajectory matrix. Moreover, it is interesting to work on the geometrical representation of this trajectory matrix rather than using directly the Frobenius norm which places us on $\mathbb{R}^{n\times p}$. Indeed, the Stiefel manifold and the unit Sphere present much better results. Finally, the unit Sphere is an interesting alternative to the Stiefel manifold, because it is only slightly less good as the Stiefel and allows more flexibility by allowing to compare Stiefel manifold elements of different dimensions.

Once the geometrical representation of the time series analyzed, we let the reader gives itself an idea on which could be the best clustering framework to use on the Stiefel manifold. In this view we have the following results:

- UMAP + HDBSCAN is the best clustering framework for 58.2% of the databases (46/79). It has the best v-measure score average of 0.323.
- K-means is able to well clusters the databases in 31.6% of the cases (25/79) and has a mean value performance of 0.267.
- hierarchical algorithm managed to cluster 29.1% of the databases (23/79) with a mean v-measure score value of 0.235.
- HDBSCAN alone obtained a ratio of 20.2% good clustering (16/79) with still less mean value of 0.212.

Part of the conclusion is that the HDBSCAN algorithm alone seems to be the worst clustering algorithm. We further point out that for nearly 16% of the databases (13/79), all 4 HDBSCAN results are zero. However, using it with UMAP seems really very interesting because the results are greatly improved. The use of UMAP as a pre-processing step of the HDBSCAN algorithm therefore seems very important. We can also notice that few parameters are used for the UMAP + HDBSCAN clustering algorithms. It was not possible to optimize those elements for all the databases, but for a user

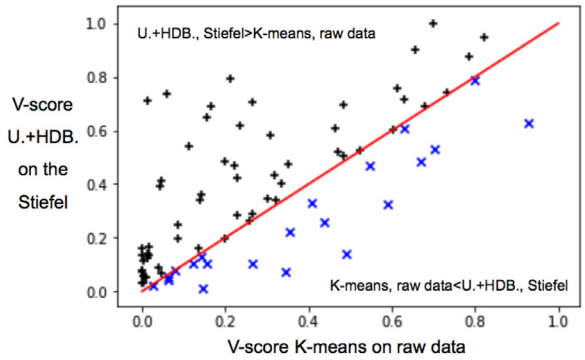


Fig. 5. Databases represented by v-measure score with UMAP + HDBSCAN on the Stiefel in function of v-measure score with K-means on raw data.

with one database, those parameters can be manipulated to have better clustering results depending on what the user is looking for.

To represent the results, we propose to compare two methods by plotting the v-measure score of one method in function of the v-measure score of the other one. The databases are represented by points, and the red line separates into two fields, when one of the two methods is better than the other. The Fig. 5 gives a visual representation of the good results obtained by UMAP+HBDSCAN on the Stiefel manifold. Indeed, when plotted against a more classic method (K-means on raw data), we clearly see the efficiency of the method.

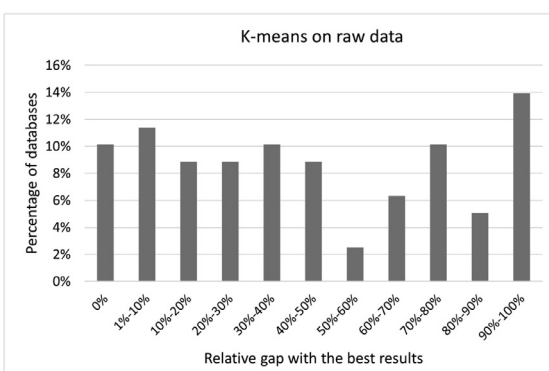
A large part of the databases are above the red line (when UMAP + HDBSCAN on the Stiefel is better than K-means on raw data). Moreover, the blue points are in general closer to the red line. Indeed, when K-means on raw data is better (this is the case for 21 databases out of 79), it is for a small amount. In average on the 21/79 databases, K-means on raw data has a higher v-measure score with 0.117. The median is even lower with a value of 0.08.

The Fig. 6 gives an other visualisation of the efficiency of UMAP + HDBSCAN on the Stiefel compared to K-means on raw data. For each databases, the relative gap $e = \frac{v_{\max} - v_{method}}{v_{\max}}$ between the results of a given method (Fig. 6a: K-means on raw data, Fig. 6b: UMAP + HDBSCAN. on the Stiefel) and the best results across the 16 methods is computed. Then, for each interval of 10% of the relative gap, the percentage of concerned databases is plotted. For example, for 10% of the databases, K-means on raw data gives results far away from the best results possible with a relative gap between 70% and 80%. This shows that UMAP + HDBSCAN on the Stiefel manifold is much more often the best method (first column). Moreover, when this method is not the best, it does not give completely wrong results either. Indeed, the number of databases falls largely when the relative gap increases. On the contrary, for the K-means method on raw data the number of databases with a very high relative gap is important, *i.e.* 14% of databases for which K-means on raw data is completely off.

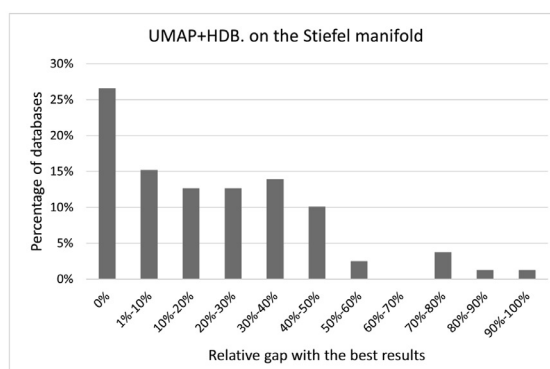
The authors of [16] performed a similar benchmark work on the same databases. They tested two distances, the Euclidean distance and the Time Wrapping distance as well as 5 clustering algorithms, the K-means, K-medoids, Fuzzy C-means, Density Peaks and Agglomerative. In addition, in their work, the authors have fixed the number of clusters of each databases with respect to the true labels given. The K-means method with the Euclidean distance on raw data appears to be the best clustering method of their benchmark (see Fig. 5). Since we show that the framework UMAP + HDBSCAN on the Stiefel manifold clearly presents better results than K-means on raw data, this framework outperforms the best one proposed in [16]. As a conclusion, none of the clustering methods proposed and tested in [16] seem better than the geometrical frameworks we propose in this work.

An other classic method to analyze time series is Soft-DTW [39,42]. We computed the clustering results obtained with this measure associated with the K-means algorithm across the 79 databases. We fond an average v-measure score of 0.272 and an average ARI score of 0.177. This obviously say that, once again, under the right choice for τ, the framework UMAP + HDBSCAN on the Stiefel manifold is clearly the better choice.

We may interpret the good results obtained when using the embedding analyzed through the Stiefel manifold by the lower sensitivity to noise corruption on the data to the delay-embedding representation. Indeed, small local perturbations due to noise have a limited impact on the global dynamic of the system. Yet, the databases available at the UCR time series srchive are extracted from real experimentation, that leads to some level of noise. Regarding the geometries, as said before, euclidean geometry directly on raw data only takes into account local differences. By using delay coordinate embedding, we look for the global dynamics of the time series. The Frobenius distance still gives limited results because we use a Euclidean metric on a particular space, not nec-



(a) Comparison between the K-means on raw data and the best method



(b) Comparison between the UMAP + HDBSCAN on the Stiefel and the best method

Fig. 6. The relative gap between the clustering of a given method and the best results is computed for each database. Then, the percentage of concerned databases by interval of relative gap is plotted for two methods: K-means on raw data, and UMAP + HDBSCAN on the Stiefel.

essarily Euclidean. Indeed, the trajectory matrices have a particular structure. Through an orthogonalization (and thus the Stiefel manifold), we can clarify the subspace generated by each trajectory matrix and use a well-defined metric that fits the geometry of the ambient space. The embedding on the Sphere, directly linked to the embedding on the Stiefel manifold, gives slightly lower results than the Stiefel manifold because of the constraint of being able to compare Stiefel manifolds of different dimensions.

6.3. Computational complexity

To complete our work, we propose a quick analysis of the computational complexity. For UMAP and HDBSCAN, an analysis is available on their respective website: https://umap-learn.readthedocs.io/en/latest/benchmarking.html, https://hdbscan.readthedocs.io/en/latest/performance_and_scalability.html. Except for PCA, UMAP was the fastest reduction of dimension algorithm. For HDBSCAN, only K-means was quicker than HDBSCAN on their benchmark. Moreover, the complexity of HDBSCAN is bounded by $\mathcal{O}(k^2)$ with k the number of elements.

However, what was really impacting was the computation of the distance, especially on the Stiefel manifold. Indeed, to carry out with our framework, it is necessary to build a pairwise distance matrix. This means, for a database of k elements, we have a complexity of $\mathcal{O}(k^2)$ for the calculation of this matrix. Moreover, the calculation time of the distance between two elements also depends on the length l of the time series. The value of l has an impact on the size of a trajectory matrix. Indeed, n and l have a linear relationship ($n = l - 1 - (m-1)\tau$). Computing the distance between random elements of $\mathbb{R}^{n\times p}$, S_n and $V_{n,p}$ for values of n ranging from 5 to 10000, one obtains a complexity of $\mathcal{O}(n)$ and thus $\mathcal{O}(l)$ for all distances. Overall, the complexity for the pairwise distance matrix is then of $\mathcal{O}(k^2 \times l)$ for a database of k elements of length l.

To give an idea, for the biggest database, the one with 8926 elements, it took us about 5 h to compute this matrix on the Stiefel manifold.

7. Conclusion

In this article, we were interested in one-dimensional time series clustering. More precisely, we proposed to embed the time series on a higher-dimensional space. We used for that the delay coordinate embedding, also known Takens' embedding, which extracts geometrical features of the times series by representing them on the so-called phase space. The induced trajectory matrices can next be analyzed under the cover of different geometries. Each geometry leads to the definition of a particular distance and a particular mean which respect the structure of this geometry. Once these elements had been determined, clustering algorithms can be applied. The good results obtained by geometric methods on high-dimensional time series, motivated us to carry out this work. We proposed to apply similar methods to one-dimensional time series using delay coordinate embedding. In addition, as the literature on one-dimensional time series clustering makes little mention of this type of methods, it appeared important to us to carry out a benchmark.

Since clustering is an unsupervised method, we tested the different geometries on databases with known 'true' labels. We started first with toy examples of time series directly extracted from dynamical system and noise free. The Stiefel manifold gave great results on those toy examples. With these promising results, we thought it was important to confirm them by carrying out a complete benchmark to determine in advance which method is the best. To do this, we used 79 databases with known expected results. These databases were extracted from the UCR archive, and are from real world example. They are therefore different from the first examples because they do not necessarily come from dynamic systems and might be corrupted by noise. We have compared 4 coupled geometries with 4 clustering algorithms. The three geometries induced by the embedding on higher-dimensional space gave significantly better results than using an Euclidean method on raw data. Moreover, the Stiefel manifold allowed better comparison and thus clustering of time series than the unit Sphere and then $\mathbb{R}^{n\times p}$.

If we look only at the clustering algorithms, a fairly clear ranking has emerged. The combination of UMAP and HDBSCAN has been the best algorithm in front of K-means, Hierarchical and HDBSCAN alone. In particular, the combination UMAP + HDBSCAN on the Stiefel manifold gave the best results. Soft-DTW also presents poorer results. This work might also give insights for work on multivariate time series. Indeed, a classic way of using delay coordinate embedding on multivariate time series is to apply the delay coordinate embedding on each channel and then to concatenate the trajectory matrices. It could be efficient to then treat the concatenation of those matrices as element on the Stiefel manifold and then apply UMAP + HDBSCAN. We can also directly use the observation matrix or the covariance matrix of the multivariate time series and analyze it through the Stiefel manifold with a QR orthogonalization.

However, two parameters came into play during the delay coordinate embedding. If m, the embedding dimension, appeared as a parameter with little influence on clustering, the τ parameter seemed to be of prime importance. For the different simulation scenarios, we determined this optimal parameter for the clustering with respect to the 'true' labels, leading consequently to a small flavor of supervision for the methods analyzed. An undergoing work is then to find an optimal way to determine τ.

In the literature, several methods are proposed to determine m and τ for the Takens' theorem. In general, for m, the false nearest neighbor method is quite widely used [40]. This is more complicated for τ. Some methods like the one proposed in [41] and which is an entropy-based method, allow to determine m and τ at the same time. However, all these methods become complicated to implement in cases of noisy and 'short' time series like the ones available at 'UCR time series'. Moreover, the best τ for the Takens' theorem is not necessarily the best for the clustering. Indeed, we do not necessarily want a perfect representation of the dynamic system attractor, but rather to highlight the difference between clusters through specific geometrical features. Determining τ properly for real time series clustering therefore remains an open question.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This study was supported by the French Ministry of Health under the project PREPS-15-000668.

Appendix

Appendix A.1
Complete results for 16 methods and 79 databases (1/3).

	Frobenius				Sphere				Stiefel				Raw data			
Databases	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.
50words	0.442	0.621	0.187	0.546	0.19	0.036	0.298	0.545	0.208	0.045	0.245	0.532	0.704	0.695	0.186	0.531
Adiac	0.162	0.06	0.306	0.208	0.186	0.005	0.311	0.58	0.137	0.092	0.465	0.651	0.154	0.06	0.315	0.534
ArrowHead	0.39	0.051	0.378	0.451	0.445	0.54	0.553	0.489	0.499	0.051	0.319	0.607	0.463	0.051	0	0.371
Beef	0.265	0.265	0.376	0.104	0.182	0.238	0.214	0.104	0.265	0.265	0.16	0.104	0.265	0.265	0.376	0.104
BeetleFly	0.174	0.206	0	0	0.356	0.399	0	0.243	0.397	0.423	0	0.347	0.302	0.206	0	0
BirdChicken	0.313	0.307	0	0.221	0.17	0.283	0	0.158	0.067	0.283	0	0.342	0.32	0.313	0	0.233
Car	0.149	0.277	0	0.139	0.448	0.576	0.44	0.303	0.391	0.04	0	0.343	0.139	0.116	0	0.303
CBF	0.649	0.211	0	0.281	0.274	0.42	0	0.274	0.283	0.431	0	0.467	0.221	0.281	0	0.264
Chlo... tion	0.005	0.015	0.018	0.021	0.005	0.015	0.02	0.012	0.018	0.032	0.029	0.03	0.005	0.006	0.023	0.017
CinC... orso	0.599	0.655	0.388	0.485	0.902	0.902	0.687	0.902	0.663	0.902	0.662	0.902	0.655	0.637	0.388	0.198
Coffee	0.812	0.524	0.476	1	1	0.812	0.871	1	0.695	0.842	0	1	0.7	0.11	0.464	0.871
Computers	0.018	0.009	0.018	0.037	0.011	0.008	0.024	0.062	0.072	0.054	0.066	0.07	0.046	0.086	0.003	0.052
CricketX	0.049	0.062	0.135	0.122	0.198	0.011	0.06	0.381	0.142	0.296	0.098	0.413	0.047	0.071	0.132	0.315
CricketY	0.174	0.095	0.167	0.11	0.119	0.005	0.14	0.311	0.091	0.021	0.102	0.364	0.141	0.174	0.162	0.191
CricketZ	0.043	0.041	0.143	0.164	0.285	0.022	0.109	0.326	0.13	0.036	0.114	0.391	0.041	0.038	0.116	0.235
Diat... tion	0.648	1	0	0.789	0.641	0.884	0	0.606	0.875	0.504	0	0.629	0.928	1	0	0
Dist... roup	0.194	0.012	0.258	0.282	0.185	0.279	0.311	0.207	0.207	0.334	0.374	0.282	0.229	0.176	0.262	0.282
Dist... rect	0.051	0.009	0.065	0.024	0.046	0.009	0.12	0.023	0.053	0.07	0.115	0.021	0.028	0.009	0.078	0.012
Dist... nxTW	0.474	0.521	0.437	0.521	0.521	0.521	0.523	0.521	0.367	0.551	0.562	0.521	0.47	0.521	0.439	0.521
Earthquakes	0.006	0.022	0.008	0.02	0	0.007	0	0.009	0.044	0.019	0.007	0.043	0.064	0.077	0.008	0.025
ECG200	0.228	0.289	0.314	0.289	0.35	0.224	0	0.268	0.242	0.319	0.239	0.423	0.23	0.289	0.299	0.185
ECG5000	0.672	0.005	0.669	0.601	0.76	0.819	0.782	0.692	0.024	0.005	0.64	0.691	0.68	0.74	0.671	0.559
ECG....ays	0	0.015	0	0.002	0.028	0.231	0	0.059	0.073	0.015	0.003	0.136	0.015	0.015	0	0.007
FaceAll	0.055	0.004	0.456	0.559	0.727	0.121	0.549	0.772	0.515	0.136	0.224	0.737	0.059	0.054	0.152	0.565
FaceFour	0.381	0.537	0	0.588	0.687	0.736	0	0.601	0.044	0.813	0	0.698	0.484	0.486	0	0.417
FacesUCR	0.115	0.022	0.252	0.416	0.251	0.102	0.253	0.637	0.209	0.102	0.185	0.607	0.629	0.058	0.326	0.188

Appendix A.2
Complete results for 16 methods and 79 databases (2/3).

	Frobenius				Sphere				Stiefel				Raw data			
Databases	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.
FISH	0.125	0.124	0.113	0.479	0.456	0.309	0.299	0.336	0.434	0.011	0.232	0.542	0.112	0.042	0.114	0.433
FordA	0	0	0.019	0.042	0.187	0.016	0.016	0.028	0.072	0.03	0.047	0.135	0	0.01	0.015	0.051
FordB	0.001	0.005	0.013	0.023	0.036	0.01	0.02	0.009	0.01	0.002	0.012	0.077	0	0.001	0.01	0.023
GunPoint	0.363	0.352	0.315	0.429	0.208	0.253	0.078	0.358	0.287	0.013	0.438	0.475	0.351	0.379	0.305	0.013
Ham	0.02	0.018	0	0.103	0.02	0.018	0	0.046	0.017	0.018	0	0.103	0.155	0.018	0	0.066
Haptics	0.098	0.017	0.07	0.156	0.079	0.017	0.081	0.182	0.018	0.017	0	0.106	0.121	0.017	0.09	0.091
Herring	0.003	0.021	0	0.005	0.017	0.021	0	0.005	0.043	0.039	0	0.03	0.001	0.021	0	0.005
InlineSkate	0.082	0.101	0.083	0.23	0.024	0.064	0.074	0.054	0.056	0.064	0.074	0.159	0.135	0.133	0.083	0.186
Inse... ound	0.52	0.548	0.36	0.293	0.475	0.423	0.288	0.29	0.2	0.324	0.279	0.467	0.547	0.557	0.357	0.552
Ital... emand	0.014	0.083	0.035	0.551	0.458	0.474	0.308	0.39	0.226	0.491	0.205	0.713	0.014	0.083	0.402	0.607
Lighting2	0.039	0.039	0	0.051	0.035	0.019	0.022	0.059	0.117	0.026	0	0.051	0.007	0.039	0	0.067
Lighting7	0.138	0.198	0.297	0.198	0.442	0.033	0.21	0.291	0.115	0.198	0.152	0.198	0.198	0.198	0.297	0.198
MALLAT	0.8	0.697	0.7	0.781	0.644	0.716	0.774	0.698	0.529	0.796	0.751	0.791	0.8	0.697	0.7	0.781
Meat	0.734	0.707	0.691	0.734	0.734	0.707	0.691	0.721	0.392	0.031	0.617	0.74	0.734	0.707	0.691	0.734
Med... ages	0.29	0.004	0.126	0.14	0.07	0.012	0.084	0.078	0.244	0.298	0.112	0.222	0.355	0.015	0.131	0.143
Midd... roup	0.019	0.112	0.139	0.129	0.051	0.139	0.139	0.139	0.007	0.113	0.078	0.139	0.013	0.111	0.139	0.111
Midd... rect	0.023	0.003	0.002	0.003	0.091	0.027	0.005	0.01	0.039	0.003	0.022	0.115	0.002	0.016	0.003	0.003
Midd... nxTW	0.461	0.466	0.454	0.506	0.506	0.506	0.506	0.506	0.485	0.424	0.477	0.506	0.484	0.47	0.443	0.506
MoteStrain	0.204	0.081	0	0.097	0.179	0.081	0	0.196	0.137	0.275	0	0.258	0.438	0.475	0	0.293
NonI... rax1	0.261	0.001	0.075	0.774	0.249	0.001	0.478	0.658	0.114	0.001	0.159	0.708	0.264	0.001	0.075	0.765
NonI... rax2	0.214	0.001	0.683	0.816	0.253	0.001	0.616	0.788	0.557	0.001	0.609	0.794	0.211	0.001	0.696	0.839
OliveOil	0.516	0.2	0.589	0.486	0.583	0.2	0.568	0.486	0.683	0.679	0.459	0.486	0.2	0.2	0.589	0.486
OSULeaf	0.072	0.322	0.273	0.317	0.372	0.485	0.44	0.402	0.263	0.39	0.248	0.434	0.318	0.424	0.304	0.285
Ohal... rect	0.005	0.009	0.065	0.003	0	0.002	0.071	0.001	0.003	0.007	0.072	0.057	0.003	0.002	0.048	0.002
Phoneme	0.505	0.5	0	0.254	0.528	0.01	0.138	0.255	0.267	0.026	0.097	0.327	0.589	0.569	0.091	0.29
Plane	0.796	0.841	0.893	0.891	0.798	0.923	0.929	0.948	0.788	0.87	0.853	0.948	0.823	0.841	0.893	0.831

Appendix A.3
Complete results for 16 methods and 79 databases (3/3).

	Frobenius				Sphere				Stiefel				Raw data			
Databases	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.	Kmeans	Hier.	HDB.	U.+HDB.
Prox... roup	0.524	0.005	0.516	0.524	0.524	0.524	0.525	0.524	0.524	0.524	0.524	0.524	0.524	0.522	0.516	0.524
Prox... rect	0.056	0.012	0.066	0.033	0.038	0.057	0.097	0.034	0.014	0.05	0.088	0.055	0.063	0.012	0.055	0.034
Prox... nxTW	0.602	0.599	0.599	0.602	0.581	0.581	0.599	0.602	0.617	0.602	0.602	0.602	0.602	0.599	0.599	0.602
Refr... ices	0.025	0.03	0.01	0.021	0.019	0.004	0.054	0.059	0.002	0.045	0.004	0.086	0.038	0.063	0.01	0.041
ShapeletSim	0.278	0.042	0	0	0.126	0.287	0	0	0.054	0.027	0	0.012	0.147	0.165	0	0
ShapesAll	0.169	0.465	0.514	0.634	0.717	0.66	0.493	0.666	0.61	0.007	0.478	0.693	0.166	0.165	0.512	0.636
Smal... nces	0.056	0.043	0.053	0.082	0.024	0.005	0.085	0.091	0.019	0.065	0.072	0.076	0.079	0.063	0.053	0.068
Sony... face	0.115	0.157	0	0.07	0.115	0.157	0	0.025	0.099	0.416	0	0.14	0.49	0.157	0	0.038
Star... rves	0.61	0.604	0.736	0.656	0.76	0.76	0.471	0.75	0.504	0.76	0.525	0.76	0.612	0.607	0.736	0.76
Strawberry	0.084	0.062	0.224	0.143	0.145	0.017	0.056	0.293	0.031	0.048	0.164	0.249	0.086	0.004	0.228	0.195
SwedishLeaf	0.234	0.042	0.218	0.55	0.262	0.008	0.357	0.631	0.356	0.008	0.313	0.62	0.234	0.004	0.218	0.546
Symbols	0	0.651	0	0	0.691	0.872	0	0.547	0.84	0.654	0	0.875	0.787	0.81	0	0.58
ToeS... ion1	0.002	0.032	0.042	0.003	0.072	0.163	0.129	0.031	0.21	0.202	0	0.132	0.144	0.055	0.01	0.035
ToeS... ion2	0.28	0.36	0.017	0.043	0.593	0.43	0	0.616	0.341	0.126	0	0.585	0.309	0.317	0.059	0.125
Trace	0.669	0.669	0.6	0.669	0.014	0.457	0.599	0.237	0.485	0.588	0.462	0.484	0.669	0.669	0.669	0.498
TwoPatterns	0.012	0.017	0.17	0.209	0.036	0.02	0.022	0.168	0	0.002	0.015	0.126	0.013	0.031	0.174	0.198
TwoLeadECG	0.086	0.086	0	0.086	0.082	0.082	0.175	0.082	0.254	0.252	0	0.197	0.086	0.086	0	0.086
uWav... aryX	0.261	0.271	0.246	0.476	0.321	0.003	0.119	0.205	0.13	0.002	0.098	0.29	0.264	0.218	0.249	0.438
uWav... aryY	0.257	0.406	0.336	0.411	0.214	0.002	0.074	0.269	0.082	0.003	0.068	0.261	0.257	0.2	0.35	0.415
uWav... aryZ	0.383	0.447	0.41	0.548	0.32	0.007	0.057	0.237	0.063	0.011	0.113	0.329	0.408	0.292	0.405	0.54
uWav... yAll	0.537	0.668	0.583	0.8	0.673	0.656	0.304	0.643	0.497	0.699	0.198	0.719	0.63	0.758	0.582	0.755
Wafer	0	0	0.225	0.143	0.263	0.006	0.268	0.143	0.11	0.042	0.214	0.161	0	0	0.257	0.123
Wine	0.001	0.091	0.019	0.034	0.193	0.091	0.016	0.009	0.087	0.091	0.004	0.071	0.344	0.091	0.001	0.023
Wor... yms	0.326	0.345	0.176	0.356	0.395	0.514	0.158	0.337	0.265	0.037	0.228	0.4	0.335	0.344	0.221	0.368
Worms	0.003	0.199	0.19	0.116	0.209	0.031	0.141	0.136	0.28	0.27	0.149	0.166	0.018	0.252	0.144	0.095
Wor... lass	0	0.061	0.092	0.004	0.086	0.037	0	0.062	0.059	0.033	0	0.076	0.001	0.08	0.032	0.009
Yoga	0.008	0.014	0.056	0.041	0.02	0	0.054	0.035	0.005	0.006	0.066	0.051	0.008	0.01	0.054	0.04

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.patcog.2021.108423.

References

[1] R. Xu, D. Wunsch, Clustering, John Wiley & Sons, 2008.
[2] G. Bouleux, Oblique projection pre-processing and TLS application for diagnosing rotor bar defects by improving power spectrum estimation, Mech. Syst. Signal Process. 41 (1) (2013) 301–312.
[3] G. Bouleux, E. Marcon, O. Mory, Early index for detection of pediatric emergency department crowding, IEEE J. Biomed. Health Inform. 19 (6) (2015) 1929–1936.
[4] M. Dugast, G. Bouleux, O. Mory, E. Marcon, Improving health care management through persistent homology of time-varying variability of emergency department patient flow, IEEE J. Biomed. Heal. Inform. 23 (5) (2019) 2174–2181.
[5] G. Li, J.J. Jung, Dynamic relationship identification for abnormality detection on financial time series, Pattern Recognit. Lett. 145 (2021) 194–199.
[6] V. Kavitha, M. Punithavalli, Clustering time series data stream - a literature survey, Int. J. Comput. Sci.Inf. Secur. 8 (1) (2010).
[7] T. Oates, L. Firoiu, P.R. Cohen, Clustering time series with hidden Markov models and dynamic time warping, in: Proceedings of the IJCAI-99 Workshop on Neural, Symbolic and Reinforcement Learning Methods for Sequence Learning, 1999, pp. 17–21.
[8] C. Pealat, G. Bouleux, V. Cheutet, Extracting Most Impacting Emergency Department Patient Flow By Embedding Laboratory-confirmed and Clinical Diagnosis on The Stiefel Manifold, Proceedings of 2019 IEEE EMBS International Conference on Biomedical Health Informatics (BHI) (2019). ISSN: 2641-3590.
[9] L. Sigal, A. Balan, M. Black, HumanEva: synchronized video and motion capture dataset and baseline algorithm for evaluation of articulated human motion, Int. J. Comput. Vis. 87 (2010) 4–27.
[10] X.S. Nguyen, L. Brun, O. Lezoray, S. Bougleux, A neural network based on SPD manifold learning for skeleton-based hand gesture recognition, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 12036–12045.
[11] P. Rodrigues, M. Congedo, C. Jutten, Multivariate time-series analysis via manifold learning, in: 2018 IEEE Statistical Signal Processing Workshop (SSP), 2018, pp. 573–577.
[12] J. Yang, J. Liang, K. Wang, P.L. Rosin, M.H. Yang, Subspace clustering via good neighbors, IEEE Trans. Pattern Anal. Mach. Intell. 42 (6) (2020) 1537–1544.
[13] R. Li, T. P. Tian, S. Sclaroff, Simultaneous Learning of Nonlinear Manifold and Dynamical Models for High-dimensional Time Series, in: 2007 IEEE 11th International Conference on Computer Vision, Rio de Janeiro, Brazil, 2007.
[14] P. Turaga, A. Veeraraghavan, A. Srivastava, R. Chellappa, Statistical computations on Grassmann and Stiefel manifolds for image and video-based recognition, IEEE Trans. Pattern Anal. Mach. Intell. 33 (11) (2011) 2273–2286.
[15] Y. Xu, S. Chen, J. Li, L. Luo, J. Yang, Learnable low-rank latent dictionary for subspace clustering, Pattern Recognit. 120 (2021) 108142.
[16] A. Javed, B. Suk Lee, D.M. Rizzo, A benchmark study on time series clustering, Mach. Learn. Appl. 1 (2020) 100001.
[17] A. Fakhrazari, H. Vakilzadian, A survey on time series data mining, in: 2017 IEEE International Conference on Electro Information Technology (EIT), 2017, pp. 476–481.
[18] J. Sun, Y. Yang, Y. Liu, C. Chen, W. Rao, Y. Bai, Univariate time series classification using information geometry, Pattern Recognit. 95 (2019) 24–35.
[19] C. O'Reilly, K. Moessner, M. Nati, Univariate and multivariate time series manifold learning, Knowl. Based Syst. 133 (2017) 1–16.
[20] G. Bouleux, M. Dugast, E. Marcon, Information topological characterization of periodically correlated processes by dilation operators, IEEE Trans. Inf. Theory 65 (10) (2019) 6484–6495.
[21] M. Dugast, G. Bouleux, E. Marcon, Representation and characterization of nonstationary processes by dilation operators and induced shape space manifolds, Entropy 20 (9) (2018) 717.
[22] J. Stark, D.S. Broomhead, M.E. Davies, J. Huke, Takens embedding theorems for forced and stochastic systems, Nonlinear Anal. 30 (8) (1997) 5303–5314.
[23] Z. Gao, N. Jin, Complex network from time series based on phase space reconstruction, Chaos 19 (3) (2009) 033137.
[24] C. Pealat, G. Bouleux, V. Cheutet, Improved Time-Series Clustering with UMAP dimension reduction method, Proceedings of 22nd International Conference on Pattern Recognition (2020).
[25] R.J.G.B. Campello, D. Moulavi, J. Sander, Density-based clustering based on hierarchical density estimates, Adv. Data Min. 7819 (2013) 160–172.
[26] L. Mc, J. Healy, J. Melville, UMAP: uniform manifold approximation and projection for dimension reduction, 2018, arXiv:1802.03426
[27] Z. Wang, T. Oates, Imaging time-series to improve classification and imputation, 24th International Joint Conference on Artificial Intelligence (2015).
[28] B. St Thomas, L. Lin, L.H. Lim, S. Mukherjee, Learning subspaces of different dimension, 2014, arXiv:1404.6841
[29] H. Karcher, Riemannian center of mass and mollifier smoothing, Comm. Pure Appl. Math. 30 (5) (1977) 509–541.
[30] J. Miao, A. Ben-Israel, On principal angles between subspaces in Rn, Linear Algebra Appl. 171 (92) (1992) 81–98.
[31] R. Zimmermann, A matrix-algebraic algorithm for the Riemannian logarithm on the Stiefel manifold under the canonical metric, 2016, arXiv:1604.05054
[32] S. Gudmundsson, An introduction to Riemannian geometry, Lecture Notes (2014) 130. https://link.springer.com/book/10.1007/978-3-319-08666-8?token=M3aff43&utm_campaign=3_fjp8312_springer_katte_M3aff43&countryChanged=true&gclid=Cj0KCQiA4b2MBhD2ARIsAIrcB-R_fZ89PB7xu-e3Bg4D5KWh3L2IJzdjAvRfabBqlBW8MfnfqeD5SK0aAsOoEALw_wcB#about.
[33] L. Kaufman, P.J. Rousseeuw, Finding Groups in Data: An Introduction to Cluster Analysis, vol. 344, John Wiley & Sons, 1990, pp. 68–125.
[34] E. Schubert, J. Sander, M. Ester, H.P. Kriegel, X. Xu, DBSCAN Revisited, revisited: why and how you should (still) use DBSCAN, ACM Trans. Database Syst. 42 (3) (2017) 1–21.
[35] H.P. Kriegel, P. Kröger, J. Sander, A. Zimek, Density-based clustering, WIREs Data Min. Knowl. Discov. 1 (3) (2011) 231–240.
[36] M. Mittal, L.M. Goyal, D.J. Hemanth, J.K. Sethi, Clustering approaches for high--dimensional databases: a review, Wiley Interdiscip. Rev. (2019).
[37] A. Rosenberg, J. Hirschberg, V-measure: a conditional entropy-based external cluster evaluation measure, in: Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), 2007, pp. 410–420.
[38] L. Hubert, P. Arabie, Comparing partitions, J. Classif. (1985).
[39] M. Cuturi, M. Blondel, Soft-DTW: a differentiable loss function for time-series, Proceedings of the 34th International Conference on Machine Learning (2017).
[40] C. Rhodes, M. Morari, False-nearest-neighbors algorithm and noise-corrupted time series, Phys. Rev. E 55 (5) (1997) 6162–6170.
[41] T. Gautama, D.P. Mandic, M.M. Van Hulle, A differential entropy based method for determining the optimal embedding parameters of a signal, in: 2003 IEEE International Conference on Acoustics, Speech, and Signal Processing, Proceedings. (ICASSP '03), vol. 6, 2003, pp. 6–29.
[42] F. Karim, S. Majumdar, H. Darabi, Adversarial attacks on time series, IEEE Trans. Pattern Anal. Mach. Intell. 43 (10) (2021) 3309–3320.

Clément Péalat obtained his engineering degree in 2018. He then continued with a thesis in the DISP laboratory. His thesis is in the medical field in partnership with the CHU of Saint-Etienne. This led him to specialize in time series clustering.

Guillaume Bouleux received the Msc. And PhD degrees in 2005 and 2007 respectively. He is the co-leader of the operation management and optimisation team of the DISP laboratory. His field of application focuses on Healthcare systems and geometry along with the topology for stochastic processes are of his principal theoretical concern.

Vincent Cheutet is full professor since 2014 at INSA Lyon. He is deputy director of the DISP Laboratory (Decision and Information System for Production). His main research topics are industrial information systems (especially PLM and ERP), knowledge management, industrial and mechanical engineering management.